

GGUSD

Smart Contract Security Assessment

June 2025

Prepared for:

Findsatoshi Lab

Prepared by:

Offside Labs

Zhoujie Tong

Tim Li





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	4
4	Key Findings and Recommendations	5
4.1	ExitFee Recipient May Never Fully Redeem Their Shares	5
4.2	Incorrect Rounding Direction Applied to feeShares	6
4.3	Informational and Undetermined Issues	8
5	Disclaimer	11



1 About Offside Labs

Offside Labs stands as a pre-eminent security research team, comprising highly skilled hackers with top - tier talent from both academia and industry.

The team demonstrates extensive and diverse expertise in modern software systems, which encompasses yet are not restricted to *browsers, operating systems, IoT devices, and hypervisors*. Offside Labs is at the forefront of innovative domains such as *cryptocurrencies and blockchain technologies*. The team achieved notable accomplishments including the successful execution of remote jailbreaks on devices like the **iPhone** and **PlayStation 4**, as well as the identification and resolution of critical vulnerabilities within the **Tron Network**.

Offside Labs actively involves in and keeps contributing to the security community. The team was the winner and co-organizer for the *DEFCON CTF*, the most renowned CTF competition in Web2. The team also triumphed in the **Paradigm CTF 2023** in Web3. Meanwhile, the team has been conducting responsible disclosure of numerous vulnerabilities to leading technology companies, including *Apple, Google, and Microsoft*, safeguarding digital assets with an estimated value exceeding **\$300 million**.

During the transition to Web3, Offside Labs has attained remarkable success. The team has earned over **\$9 million** in bug bounties, and **three** of its innovative techniques were acknowledged as being among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



2 Executive Summary

Introduction

Offside Labs completed a security audit of GGUSD smart contracts, starting on June 09, 2025, and concluding on June 13, 2025.

Project Overview

GGUSD (Good Game US Dollar) is FSL's first stablecoin, designed to enable seamless and rewarding transactions for both players and developers within and beyond the FSL ecosystem.

Issued in partnership with Agora and backed by U.S. Treasury assets, GGUSD is a USD-pegged stablecoin that combines price stability with on-chain yield. It offers users passive rewards without the need for staking or lock-in periods, making it the first flexible stablecoin of its kind.

As a gaming-focused token at its core, GGUSD is intended to serve a wide range of utility scenarios across the Web3 gaming ecosystem, both within FSL products and in the broader market.

Audit Scope

The assessment scope contains mainly the smart contracts of the GGUSD program for the GGUSD project.

We listed the files we have audited below:

- GGUSD
 - GGUSD/ERC4626Fees.sol
sha256: c5f08ab9153f2a18c29ceb37bec28c3945170e659fb87c9435c951418d139751
 - GGUSD/GGUSD.sol
sha256: 5d03dd7c6398367f4bced8f720990fe9326075bf672ac80c510c7449666e8413
 - LayerZeroOFT/GGUSDOFTAdapter.sol
sha256: f0e6bcf96c0481ded4a426f4e5c6ce59af10db60a65516035f70e3f034ba048b
 - LayerZeroOFT/GGUSDOFT.sol
sha256: 849edae5b769b872c3b9edd8f3a123e2b91d15cfbc8a04f72c02fdac8742294f

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 2 low issues
- 4 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	ExitFee Recipient May Never Fully Redeem Their Shares	Low	Fixed
02	Incorrect Rounding Direction Applied to feeShares	Low	Fixed
03	Incorrect feeAssets Calculation in Deposit Event During Exit Fee Collection	Informational	Fixed
04	Unnecessary Check in _debit Function	Informational	Acknowledged
05	Non-Standard Initialization Method __OFT_init and __OFTAdapter_init	Informational	Acknowledged
06	Function maxWithdraw Should Be Overridden to Handle Fee Logic	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 ExitFee Recipient May Never Fully Redeem Their Shares

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In `ERC4626Fees.sol`, when the protocol charges fees (on either `deposit` or `redeem`), it mints fee shares to the configured fee recipient. These fee shares are calculated using `_feeOnRaw(...)`, which always rounds up due to `Math.Rounding.Ceil`.

This introduces a subtle issue: when the fee recipient later tries to redeem their accumulated shares, a small portion of their balance will always remain unredeemable due to the same rounding logic applying in reverse when calculating fees during redemption.

Since exit fees are also taken in shares, any attempt by the fee recipient to redeem their entire balance will result in:

- A fee being calculated (on their full share balance),
- Then that fee being subtracted from their balance before asset withdrawal,
- Resulting in fewer assets than expected,
- And leaving a small amount of shares behind.

Impact

The fee recipient (e.g., the protocol owner or treasury) will never be able to redeem their entire share balance, regardless of how small it gets. A residual amount of protocol-owned shares will always remain stuck in the vault.

While this doesn't affect user funds or the core protocol functionality, it may lead to accounting inaccuracies, confusion in audits, or difficulty fully emptying the vault during migrations.

Proof of Concept

Assume the following:

- The vault uses a 1% exit fee (`_exitFeeBasisPoints() = 100`),
- The fee recipient owns exactly 100 shares (earned via prior fee minting),
- The recipient attempts to redeem all `100` shares.

Here's what happens step by step:

1. Exit fee is calculated using `_feeOnRaw(100, 100)`:

```
feeShares = ceil(100 * 100 / 10,000) = ceil(1) = 1
```



2. The redeemable shares are:

```
redeemShares = 100 - 1 = 99
```

3. The fee recipient receives assets corresponding to 99 shares, and 1 share remains in the vault.

4. If the recipient tries to redeem the remaining 1 share, the exit fee becomes:

```
feeShares = ceil(1 * 100 / 10,000) = ceil(0.01) = 1
```

5. Now, `redeemShares = 1 - 1 = 0` , so no assets are received, and the share is still stuck.

Recommendation

One of the following mitigations can be considered:

- Option 1: Exclude fee recipient addresses from fee deductions during `redeem` or `withdraw` .
- Option 2: Allow exact/full redemption if the caller is the fee recipient and attempts to redeem their full balance.

Mitigation Review Log

Fixed in file GGUSD/ERC4626Fees.sol

(sha256: 4b2619fff7b8c53110091207ec8f17b539b7d9035856a130d76e596c0f034762)

4.2 Incorrect Rounding Direction Applied to feeShares

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision Error

Description

In the `_deposit` function, `super.previewDeposit` is used to calculate `feeShares` based on `feeAssets` , and this operation applies rounding down. However, the `_deposit` function handles both the `mint` and `deposit` logics. When processing a `mint` operation, applying rounding down to calculate `feeShares` may result in an underestimation of `feeShares` , which also leads to inconsistencies with the `feeShares` calculated in `previewMint` .

Impact

This issue may result in the recipient collecting fewer fees than the amount actually deducted from the user. Additionally, the values emitted in the event logs may be inaccurate,



potentially misleading off-chain systems or observers that rely on these logs. Furthermore, the total supply of shares may be lower than expected, which could affect the accuracy of the protocol's accounting and any functionality that depends on the total share supply.

Proof of Concept

First, it should be noted that the Vault's totalAssets can increase through direct transfer operations. Therefore, it is entirely possible for totalAssets to be greater than totalSupply.

Assume the following conditions:

- Fee rate: 1%
- Vault totalAssets: 201
- Vault totalSupply: 200
- The user intends to mint 90 shares.

In previewMint, the calculation is as follows:

```
feeShares = ceil(90 * 100 / (10000 - 100)) = ceil(0.9091) = 1
assets = ceil((90 + 1) * 201 / 200) = ceil(91.455) = 92
```

In _deposit, the calculation is:

```
feeAssets = ceil(92 * 100 / 10000) = ceil(0.92) = 1
feeShares = floor(1 * 200 / 201) = floor(0.995) = 0
```

As shown above, an inconsistency occurs: previewMint calculates feeShares as 1, but _deposit calculates it as 0. This demonstrates that the feeShares calculated in _deposit can be lower than expected, even reaching zero, which is inconsistent with previewMint. This mismatch may lead to unexpected results in fee collection logic.

Recommendation

It is recommended to separate the handling logic for `deposit` and `mint` operations to make the function behavior more explicit and maintainable. This would help prevent inconsistencies arising from applying the same rounding approach to both cases, despite their different requirements.

As a less intrusive alternative, it is suggested to explicitly use `_convertToShares(assets, Math.Rounding.Ceil)` when calculating feeShares. This would clearly specify the intended rounding direction, improve code readability, and reduce the potential for misunderstandings or miscalculations.

Mitigation Review Log

Fixed in file GGUSD/ERC4626Fees.sol

(sha256: 4b2619fff7b8c53110091207ec8f17b539b7d9035856a130d76e596c0f034762)



4.3 Informational and Undetermined Issues

Incorrect feeAssets Calculation in Deposit Event During Exit Fee Collection

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Precision Error

In the `_withdraw` function of `ERC4626Fees.sol`, when the protocol collects exit fees during a `redeem` operation, the `feeAssets` field emitted in the `Deposit` event is calculated as:

```
uint256 feeAssets = super.previewRedeem(feeShares);
```

However, this value does not always accurately represent the actual assets corresponding to the `feeShares`. Here's why:

When a user redeems `shares`, the fee taken is:

```
feeShares = _feeOnRaw(shares, _exitFeeBasisPoints());
```

And the user receives:

```
assets = super.previewRedeem(shares - feeShares);
```

To correctly compute the amount of assets corresponding to the `feeShares`, we should consider the difference between the assets for the full `shares` and the remaining `shares - feeShares`:

```
feeAssets = super.previewRedeem(shares) - super.previewRedeem(shares - feeShares);
```

However, the current implementation simplifies this to:

```
feeAssets = super.previewRedeem(feeShares);
```

This is not always accurate because `previewRedeem` is non-linear in general (due to possible rounding). This leads to a subtle discrepancy in the `feeAssets` value reported in the `Deposit` event, which may cause issues in accounting, analytics, or frontend display.

When user `Deposit`, there is same problem.

Mitigation Review Log:

Partially fixed in file GGUSD/ERC4626Fees.sol

(sha256: 4b2619fff7b8c53110091207ec8f17b539b7d9035856a130d76e596c0f034762)

Fixed in file GGUSD/GGUSD.sol

(sha256: 41310b263ad5ff0963585faffb9935833d54eafcb775139a2d733f967cfd5fe0)



Unnecessary Check in `_debit` Function

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code Quality

In the files `GGUSDOFT.sol` and `GGUSDOFTAdapter.sol`, the `_debit` function includes the following check:

```
if (amountReceivedLD > amountSentLD) {
    revert("Amount received cannot be greater than amount sent");
}
```

However, this check appears to be unnecessary, as `amountReceivedLD` is guaranteed to be less than or equal to `amountSentLD` under all expected conditions.

- When `amountReceivedLD` is obtained via `_debitViewNoFee`, it is computed as `amountReceivedLD = _removeDust(_amountLD)`. Since `_removeDust` reduces the amount by removing rounding dust, it follows that `amountReceivedLD <= _amountLD`, and by definition `_amountLD == amountSentLD`.
- When derived from `_debitView`, the computation is `amountReceivedLD = _removeDust(_amountLD - fee)`, which also ensures `amountReceivedLD <= _amountLD`. Again, `amountSentLD == _amountLD`, so the inequality holds.

Non-Standard Initialization Method `__OFT_init` and `__OFTAdapter_init`

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code Quality

According to the official usage examples provided by LayerZero Labs, the `initialize` function should explicitly call `__OFTFee_init` and `__OFTAdapterFee_init` respectively.

```
__OFT_init(_name, _symbol, _delegate);
__Ownable_init(_delegate);
feeAddress = address(0);
```

`GGUSDOFT.sol`

```
__OFTAdapter_init(_delegate);
__Ownable_init(_delegate);
feeAddress = address(0);
```

`GGUSDOFTAdapter.sol`

Function `maxWithdraw` Should Be Overridden to Handle Fee Logic

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error



According to EIP-4626, the return value of `maxWithdraw` MUST NOT exceed the actual amount that can be withdrawn from the contract. Therefore, when implementing this function, any applicable fees must be properly accounted for by deducting them from the returned value.

Failing to consider fees in the `maxWithdraw` calculation may result in the function returning a value higher than what the contract will actually allow to be withdrawn. This could cause unexpected errors or reverts when external systems, relying on the ERC-4626 standard, attempt to interact with the contract based on the reported maximum withdrawable amount.



5 Disclaimer

This report reflects the security status of the project as of the date of the audit. It is intended solely for informational purposes and should not be used as investment advice. Despite carrying out a comprehensive review and analysis of the relevant smart contracts, it is important to note that Offside Labs' services do not encompass an exhaustive security assessment. The primary objective of the audit is to identify potential security vulnerabilities to the best of the team's ability; however, this audit does not guarantee that the project is entirely immune to future risks.

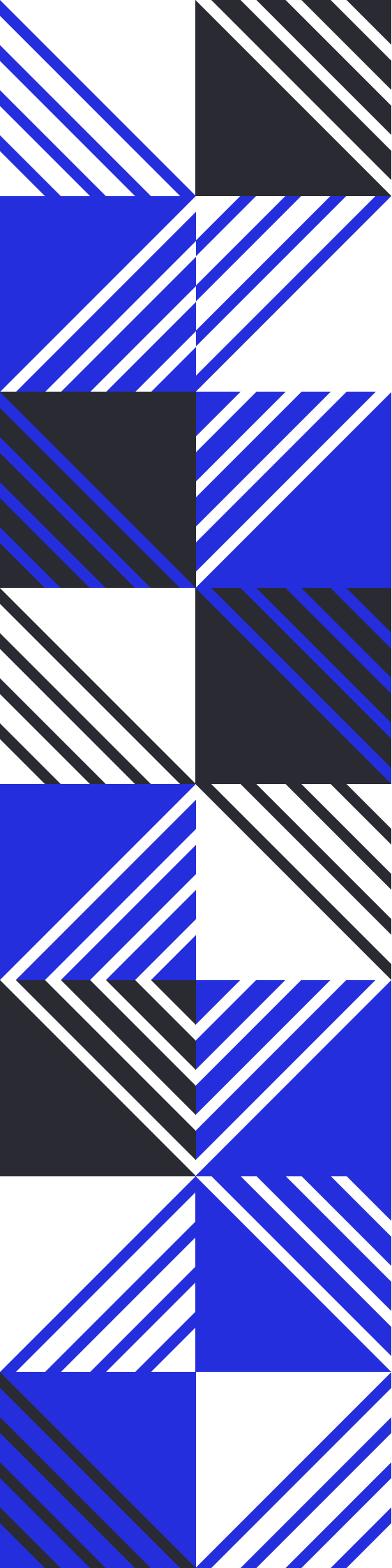
Offside Labs disclaims any liability for losses or damages resulting from the use of this report or from any future security breaches. The team strongly recommends that clients undertake multiple independent audits and implement a public bug bounty program to enhance the security of their smart contracts.

The audit is limited to the specific areas defined in Offside Labs' engagement and does not cover all potential risks or vulnerabilities. Security is an ongoing process, regular audits and monitoring are advised.

Please note: Offside Labs is not responsible for security issues stemming from developer errors or misconfigurations during contract deployment and does not assume liability for centralized governance risks within the project. The team is not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By utilizing this report, the client acknowledges the inherent limitations of the audit process and agrees that the firm shall not be held liable for any incidents that may occur after the completion of this audit.

This report should be considered null and void in case of any alteration.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs